CS316 – DevOps Lab Exercises

Lab # 03

REG#: _____ NAME: _____

COURSE CODE: CS316 INSTRUCTOR: MUHAMMAD SAJID ALI

TOTAL MARKS: Non-Graded

TASK 1 – Automate directory backup using conditionals, and exit codes.

1. Name script `backup.sh`.
2. Ask user to input a directory path.
3. Create a backup file (`backup_<timestamp>.tar.gz`).
4. Check if `tar` succeeded:
    a. Print success or failure message.
    b. Log status and timestamp to backup.log.

Expected Output: .tar.gz file and backup.log.

TASK 2 – Create a Bash script that automate the setup of Apache virtual host for a website.

1. Accepts domain (or directory) name as input
2. Creates directory in /var/www/
3. Creates config file
4. Creates basic index.html
5. Reloads server
6. Check the newly deployed website

TASK 3 – Create a Bash script named `wserver.sh` that takes a `command-line argument` to either

- Install `nginx` web server if not already installed (`-i` → `for installation`)
    - If nginx is already installed, print a message and `exit` successfully.
    - Otherwise, install nginx using `apt` and log the output to a file.
    - Show a `success` or `error` message based on whether the installation was successful.
- Remove `nginx` if it is already installed (`-r` → `for removal`)
    - If nginx is not installed, print a message and exit successfully.
    - Otherwise, remove nginx using apt and log the output to a file.
    - Show a success or error message after removal.
- Redirect all install/remove command output to a log file (e.g., nginx.log)
- Exit with `0 on success`, `1 on failure` or incorrect usage.
- Display help message if no or `incorrect flag` is passed.
- Check the status of the nginx server
    - You may need stop apache2 if required
    - Please note by default nginx also uses /var/www/html as root directory

TASK 4 – Combine all provided Lab scripts into a single reference file

- Finds all `.sh` files using `for loop` in the current directory (or parent directory if needed)

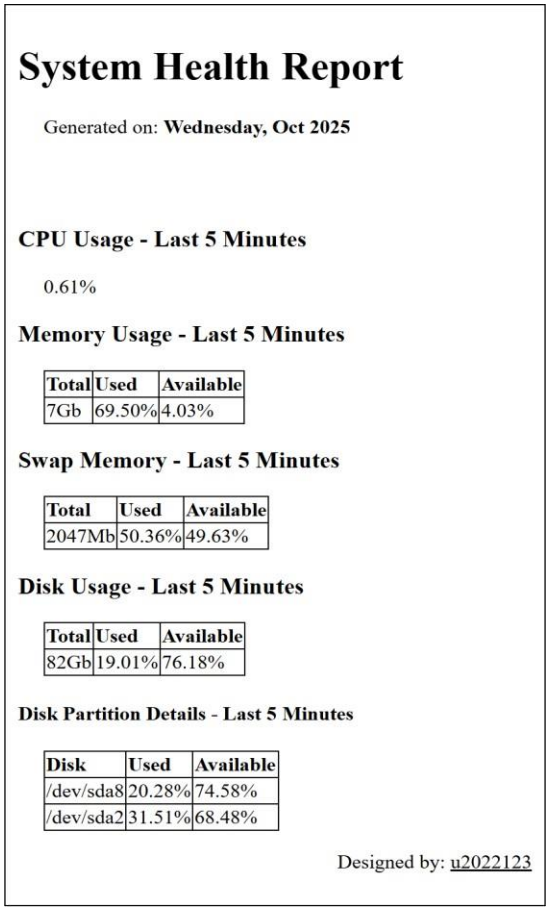
- Combines them into a single file named `lab_reference.sh`
- Adds a clear separator for each script using its file name
- Appends a blank line after each file's contents for readability

TASK 5 – System Health Report

Create a Bash script that collects key system information and writes it to an HTML report.

1. Name the file `system_health.sh`.
2. Collect and display:
    a. Current date as shown in figure
    b. Memory usage as shown in the give figure [Hint: use awk utility along with other commands]
        i. Total memory and swap memory
        ii. Used memory and swap memory in percent
        iii. Available memory and swap memory in percent
    c. Disk usage [Hint: use regex pattern in awk to filter output records, ! used to negate & ^ means to start a string with chars]
        i. Total disk space
        ii. Total disk space used and available
        iii. List of partitioned disks available, do not include *tmpfs* disks by using --exclude flag
            1. Show disk name
            2. Disk space used
            3. Disk space available
3. Use heredoc to write this info into `system_health.html`.
4. Host this `system_health.html` as `index.html` using `nginx` server.
5. If successful, print success otherwise print an error and exit with code 1.

Expected Output: HTML file with formatted system info. Sample `system_health.html` is available on the link already shared.



System Health Report

Generated on: **Wednesday, Oct 2025**

CPU Usage - Last 5 Minutes

0.61%

Memory Usage - Last 5 Minutes

Total	Used	Available
7Gb	69.50%	4.03%

Swap Memory - Last 5 Minutes

Total	Used	Available
2047Mb	50.36%	49.63%

Disk Usage - Last 5 Minutes

Total	Used	Available
82Gb	19.01%	76.18%

Disk Partition Details - Last 5 Minutes

Disk	Used	Available
/dev/sda8	20.28%	74.58%
/dev/sda2	31.51%	68.48%

Designed by: u2022123

Note: You may please refer to notebook and presentation slides if required.